

Relatório Técnico — Vitrine do Largo da Ordem (V1)

Vitrine Virtual — Feira do Largo da Ordem

Projeto da disciplina **Mobile Development iOS — PUCPR (2026)** — Avaliação Somativa SwiftUI. App iOS nativo em **SwiftUI** que simula uma vitrine virtual dos produtos artesanais vendidos na tradicional feira de domingo do Largo da Ordem, em Curitiba. O foco principal do projeto é a implementação rigorosa das diretrizes de **Acessibilidade (A11y)** da Apple.

*Este projeto foi evoluído de **MVP de vitrine** para **e-commerce funcional** com autenticação (Supabase Auth), múltiplos endereços via ViaCEP, checkout em 4 etapas, lista de pedidos com timeline de status, política de privacidade/termos LGPD-compliant, design system próprio, CI no GitHub Actions e 42 testes automatizados. Toda a evolução está documentada em `PLANO_ECOMMERCE.md` (PLANO_ECOMMERCE.md) e nos ADRs em `docs/adr/` (docs/adr/).*

Links da entrega

- **Vídeo de apresentação (YouTube — Não Listado):** (adicionar link após gravação)
- **Repositório do projeto:**
https://github.com/LucasStop/ios_development/tree/main/Somativa3/VitrineLargoOrdem (https://github.com/LucasStop/ios_development/tree/main/Somativa3/VitrineLargoOrdem)

Componentes do grupo

- Lucas Stopinski da Silva
- Lucas Bruno e Silva

Como rodar

1. Abrir `VitrineLargoOrdem.xcodeproj` no Xcode 15+ (testado com Xcode 26).
2. Selecionar um simulador iPhone ou iPad (iOS 17+).
3. `Cmd+R` para executar.

O projeto é gerado a partir de `project.yml` via `xcodegen` (<https://github.com/yonaskolb/XcodeGen>). Para regenerar: `xcodegen generate` na raiz do projeto.

Arquitetura

MVVM + Repository pattern + Design System + Composition Root (`AppDependencies`).

Organização por *feature* conforme `ADR-0001` (<docs/adr/0001-stack-decisions.md>):

```
VitrineLargoOrdem/
├── App/
│   ├── VitrineLargoOrdemApp.swift - @main, registra AppDependencies + AuthGate
│   ├── AppDependencies.swift     - Composition root (factories de ViewModels)
│   ├── AuthGate.swift            - Roteia entre Login e RootTabView
│   └── RootTabView.swift          - 4 abas: Vitrine, Favoritos, Carrinho, Perfil
├── DesignSystem/
│   ├── Tokens/                   - DSColor, DSFont, DSSpacing
│   └── Components/               - CategoryChipsView, SkeletonCardView
├── Core/
│   ├── Formatters/PrecoFormatter - Preço visual + acessível
│   ├── Persistence/              - ModelContainer SwiftData (8 entidades)
│   ├── Repositories/             - ProductRepository, FavoriteRepository,
│   │                               CartRepository, AddressRepository, OrderRepo
│   ├── Services/ViaCEPService    - Cliente HTTP do ViaCEP
│   └── Supabase/                 - AppConfig + SupabaseClient (REST/Auth)
└── Features/
    ├── Vitrine/                  - Catálogo, busca, favoritos
    ├── Carrinho/                 - Cart + CartItem persistente
    ├── Favoritos/                - Lista cruzada Produto × FavoriteItem
    ├── Identidade/               - Auth, Perfil, Endereços
    ├── Checkout/                 - Fluxo 4 etapas com pagamento mock
    ├── Pedidos/                  - Lista + Timeline com simulação local
    ├── Onboarding/               - 3 telas iniciais
    └── Sobre/                    - Política, Termos, Sobre o app
```

- **Models** = `@Model` SwiftData puros, sem UIKit/SwiftUI.
- **Repositories** abstraem a fonte (SwiftData local hoje, Supabase em paralelo).
- **ViewModels** `@MainActor` recebem repositories via DI no init; expõem estado `@Published`.
- **Views** consomem ViewModels via `@StateObject` / `@ObservedObject` e propagam ações por closures ou métodos.

Acessibilidade (A11y) — destaque do projeto

Requisito	Implementação
VoiceOver — imagens	<code>.accessibilityLabel("Imagem ilustrativa de \\$(nome), produzida por \\$(artesão)")</code> em todas as imagens dos produtos
VoiceOver — preços	Propriedade <code>precoAcessivel</code> no model retorna <code>"Preço: 45 reais"</code> em vez de <code>"R cifrão 45 ponto 00"</code>
Touch targets	<code>BotaoFavoritoView</code> força <code>.frame(minWidth: 44, minHeight: 44)</code> + <code>.contentShape(Rectangle())</code> para área clicável completa
Dynamic Type	Toda tipografia usa fontes semânticas (<code>.headline</code> , <code>.body</code> , <code>.subheadline</code>); <code>@ScaledMetric</code> no tamanho das imagens dos cards; nenhum <code>.frame(height:)</code> fixo em textos
Ordem de leitura nos detalhes	<code>.accessibilitySortPriority</code> ordena: Nome (10) → Metadados (8) → Descrição (6) → Imagem (5) → Botão de contato (4)
Estado de favorito	Label dinâmico: "Adicionar X aos favoritos" / "Remover X dos favoritos", com <code>.symbolEffect(.bounce)</code> na transição
Hints contextuais	<code>.accessibilityHint("Toque duplo para ver detalhes")</code> no link do card
Estado vazio	<code>ContentUnavailableView</code> nativa, totalmente acessível por padrão

Decisões de UI/UX

- **SwiftUI puro, sem UIKit:** usa apenas APIs nativas, garantindo aderência total ao Dynamic Type e ao Modo Escuro sem código extra.
- **LazyVGrid adaptativo:** `GridItem(.adaptive(minimum: 150), spacing: 16)` dá **2 colunas** no iPhone retrato, **3** no iPhone landscape ou no iPad, **4+** no iPad Pro horizontal. Tudo sem device check.
- **Imagens via SF Symbols:** cada produto tem um símbolo coerente com sua categoria (`leaf.fill` para madeira, `fork.knife` para comidas, `paintbrush.fill` para arte, etc.). Vantagens: sem peso no bundle, escala perfeitamente no Dynamic Type, segue a paleta do sistema, suporta acessibilidade nativa.
- **Gradientes leves de accent color** nos cards: dão personalidade sem competir com o conteúdo. Funciona bem em claro e escuro.
- **NavigationStack + .searchable embutida na barra:** padrão idiomático moderno do SwiftUI 5 (iOS 17+).

- **Botão de favoritar com `.symbolEffect(.bounce)`** : micro-animação que confirma a ação para usuários videntes, sem interferir no VoiceOver.
- **Tela de detalhes com seções cartonadas**: agrupa metadados (artesão/categoria/preço), descrição e ação principal em "cards" visuais, facilitando varredura visual e leitura linear pelo VoiceOver.

Decisões de implementação

- **ViewModel com `@MainActor`** : evita warnings de concorrência no Swift 6 e garante que mutações de `@Published` rodem na main thread.
- **`produtosFiltrados` como computed property**: derivada de `produtos + termoBusca`, recalcula apenas quando algum deles muda. Evita armazenar estado duplicado.
- **`@ScaledMetric` para altura de imagens**: as imagens crescem proporcionalmente quando o usuário aumenta a fonte do sistema, mantendo a hierarquia visual.
- **Botão favoritar com `.buttonStyle(.plain)`** : necessário para o botão funcionar dentro de um `NavigationLink` sem disparar o link ao clicar no botão.
- **`accessibilityHint` separado do `accessibilityLabel`** : o label descreve o que o elemento é; o hint descreve o que acontece ao tocar. Padrão recomendado pela Apple.

Funcionalidades

Requisitos obrigatórios — atendidos

-  `struct ProdutoArtesanal: Identifiable` com todos os 8 campos exigidos
-  `LazyVGrid` dentro de `ScrollView` com `GridItem(.adaptive(minimum: 150))`
-  Card com imagem, nome, preço formatado e botão favoritar
-  `NavigationLink` para tela de detalhes (exceto ao tocar no botão favoritar)
-  Tela de detalhes completa + botão "Entrar em contato com o Artesão"
-  `@StateObject` + `@Published` para estado reativo dos favoritos
-  `.searchable` filtrando por nome ou categoria

Acessibilidade — atendida

-  `accessibilityLabel` descritivo em todas as imagens
-  Preço acessível (sem leitura literal de "R\$" e "ponto zero zero")
-  Touch targets de 44×44 no botão favoritar
-  Dynamic Type funcional em toda a UI
-  `accessibilitySortPriority` na tela de detalhes (Nome antes da Imagem)

Produtos incluídos

Produto	Categoria	Preço
Escultura de Capivara em Madeira	Madeira	R\$ 85,00
Quibe Frito Tradicional	Comidas	R\$ 12,00
Tela "Calçadas de Curitiba"	Arte	R\$ 250,00
Manta de Tricô Colorida	Vestuário	R\$ 95,00
Relógio de Bolso Antigo	Antiquidades	R\$ 180,00
Cuia de Mate Esculpida	Madeira	R\$ 45,00
Boneca de Pano Maria	Vestuário	R\$ 35,00
Geleia Artesanal de Pinhão	Comidas	R\$ 22,00
Bijuteria com Pedras do Paraná	Acessórios	R\$ 60,00
Vaso de Cerâmica Pintado	Arte	R\$ 75,00
Cinto de Couro Trabalhado	Vestuário	R\$ 110,00
Sabonetes Naturais Trio	Beleza	R\$ 28,00

Dificuldades encontradas

- **Botão dentro de NavLink:** por padrão, o `NavLink` consome o toque em qualquer subview do seu label, fazendo com que tocar no coração também navegasse para a tela de detalhes. A primeira solução com `.buttonStyle(.plain)` em ambos não isolava o tap. Resolvido usando `.buttonStyle(.borderless)` no botão de favoritar — esse estilo permite que o botão filho receba o toque independentemente do container clicável pai, comportamento idiomático do SwiftUI para botões em listas/cards.
- **Estado de favorito não atualizava na tela de detalhes:** passar o produto como cópia por valor (`let produto: ProdutoArtesanal`) congelava o estado no momento da navegação — tocar no coração da toolbar dos detalhes alterava o ViewModel, mas a tela continuava lendo da cópia local. Resolvido recebendo o `viewModel` como `@ObservedObject` e o `produtoId: UUID`, lendo o produto atual via `viewModel.produto(comId:)` sempre que o `body` recompila.
- **Leitura do preço pelo VoiceOver:** ao usar `Text(produto.precoFormatado)`, o VoiceOver lia "R cifrão 45 ponto 00". Resolvido criando a propriedade `precoAcessivel` no model que

retorna texto natural ("Preço: 45 reais") e aplicando

`.accessibilityLabel(produto.precoAcessivel)` no texto.

- **Touch target real:** usar apenas `.frame(minWidth: 44, minHeight: 44)` não basta — a área clicável fica limitada ao ícone. Resolvido adicionando `.contentShape(Rectangle())` para que o tap seja capturado em todo o frame.
- **Ordem de leitura nos detalhes:** o enunciado pede que o Nome seja lido **antes** da Imagem. Usar `.accessibilitySortPriority(10)` no Nome e `(5)` na Imagem inverte a ordem visual sem precisar reorganizar o layout.
- **Dynamic Type em imagens fixas:** imagens com `.frame(height: 130)` ficavam pequenas demais quando o usuário aumentava a fonte. Solução: trocar por `@ScaledMetric(relativeTo: .body) private var alturaImagem: CGFloat = 130`, que escala junto com o tipo do sistema.
- **NumberFormatter recriado por célula:** a primeira versão instanciava `NumberFormatter` dentro do computed property, criando um objeto por acesso. Em um grid com 12+ cards, isso era custoso. Resolvido extraíndo para `private static let precoFormatter` no struct — uma única instância compartilhada.
- **SF Symbol inexistente:** a primeira versão usava `belt` para o cinto de couro, símbolo que não existe no catálogo do iOS — apareceria um placeholder vazio no card. Trocado por `bag.fill`, semanticamente apropriado para acessório.

Roteiro do vídeo

O roteiro detalhado da apresentação está em `ROTEIRO_APRESENTACAO.md` (`ROTEIRO_APRESENTACAO.md`), com divisão de falas entre Lucas Stopinski e Lucas Bruno, timestamps, o que mostrar em cada momento e demos específicos de acessibilidade (VoiceOver + Dynamic Type).

Evolução para e-commerce completo

O plano técnico-estratégico para transformar este projeto somativo em um marketplace funcional está em `PLANO_ECOMMERCE.md` (`PLANO_ECOMMERCE.md`) (também em PDF: `Plano_Evolucao_Ecommerce.pdf` (`Plano_Evolucao_Ecommerce.pdf`)). O documento foi construído a partir de brainstorm em 5 domínios (71 features mapeadas) seguido de crítica arquitetural e síntese executável, contendo:

- 6 princípios norteadores
- 14 decisões técnicas estratégicas com justificativa (Supabase, SwiftData, Mercado Pago, Nuke, Sentry, etc.)
- Roadmap em 4 fases (MVP / V1 / V2 / V3) com tabelas detalhadas por feature
- Mudanças de arquitetura no código atual (estrutura de pastas, refactors)

- Top 5 riscos com mitigação
- 5 próximos passos imediatos

CI/CD

O projeto tem CI rodando via GitHub Actions em todo push e PR para `main` e branches `feature/**`:

- **build-and-test** (macOS): xcodegen → build → 32 testes unitários (Swift Testing) → 10 UI Tests (XCUITest)
- **lint** (Ubuntu): SwiftLint com `.swiftlint.yml` do projeto

Detalhes em `.github/workflows/README.md` (`../../github/workflows/README.md`). Configuração no `ios.yml` (`../../github/workflows/ios.yml`).

Backend (Supabase) — opcional

O app funciona **100% offline**. Quando `USE_SUPABASE=1` (default), também sincroniza com o backend:

- **URL e chave** já configuradas em `AppConfig.swift` (`VitrineLargoOrdem/Core/Supabase/AppConfig.swift`) — sobrescreva via env vars `SUPABASE_URL` e `SUPABASE_ANON_KEY` no CI.
- **Schema SQL** para rodar no SQL Editor do dashboard: `docs/supabase/01-schema.sql` (`docs/supabase/01-schema.sql`) (idempotente; cria 4 tabelas, 2 ENUMs, triggers e Row-Level Security).
- **Guia passo a passo**: `docs/supabase/README.md` (`docs/supabase/README.md`).
- **Fallback automático**: em qualquer erro de rede, o `SupabaseAuthService` recai no `LocalAuthService` sem perder UX (princípio offline-first).
- **Sem Apple Developer**: removemos Sign in with Apple para não exigir o programa pago. Email/senha + modo convidado cobrem o MVP.

Funcionalidades do app

Tela	O que faz
Onboarding	3 telas explicativas na primeira execução; flag em <code>@AppStorage</code> .
Login / Cadastro	Email/senha com validação de regex, modo convidado, animação no AuthGate.
Vitrine	LazyVGrid responsivo (2-4 colunas), chips de categoria, busca por nome/categoria, skeleton loading, empty state com botão "limpar filtros".
Detalhe do produto	Imagem ampliada, preço acessível, estoque/disponibilidade, botão "Adicionar ao carrinho", ShareLink, contato com artesão.
Favoritos	Lista cruzada com a vitrine; mesmo card; sincroniza ao remover/adicionar.
Carrinho	Add/remove/qty, resumo (subtotal + frete + total), botão "Ir para o checkout".
Checkout 4 etapas	Endereço → Pagamento → Revisão → Confirmação, com barra de progresso animada, aceite dos termos e número de pedido (VLO-XXXXXX).
Perfil	Avatar via PhotosPicker (redimensionado a 512px), edição inline de nome, atalho para endereços e pedidos, botões Sair e Excluir conta (LGPD).
Endereços	CRUD com swipe-actions (editar/excluir/padrão), busca automática por CEP via ViaCEP, autocomplete de logradouro/bairro/cidade/UF.
Pedidos	Lista com badge colorido por status; detalhe com timeline visual (recebido → produção → despachado → entregue) que avança automaticamente a cada 12s para a demo.
Sobre / Política / Termos	Documentos LGPD-compliant integrados no app.

Testes automatizados de acessibilidade

O target `VitrineLargoOrdemUITests` tem **10 testes XCUITest** que validam todos os requisitos de A11y do enunciado:

- Labels descritivos nas imagens (não genéricos)
- Preço lido como "Preço: 85 reais" (não "R cifrão")
- Touch target $\geq 44 \times 44$ no botão favoritar
- Navegação para tela de detalhes
- **Tap no favoritar NÃO dispara navegação** (validação do fix `.borderless` + `@ObservedObject`)

- Busca por nome e por categoria
- `ContentUnavailableView` quando busca não retorna nada

Rodar: `xcodebuild test -project VitrineLargo0rdem.xcodeproj -scheme VitrineLargo0rdem -destination 'platform=iOS Simulator,name=iPhone 17' -only-testing:VitrineLargo0rdemUITests`

Vídeo de apresentação

(adicionar link do YouTube — não listado, 10–20min)